

Mini Libreria per Sistemi Lineari

Metodi iterativi per matrici simmetriche e definite positive

Nicolas Chines (matr. 899536)

Jacopo Borgato (matr. 866305)

Corso di Metodi per il Calcolo Scientifico

Università degli Studi di Milano-Bicocca

Anno Accademico 2025-2026

12 maggio 2026

Sommario

Il progetto consiste nella realizzazione di una libreria Python per la risoluzione di sistemi lineari con matrice simmetrica e definita positiva (SPD). Sono stati implementati quattro metodi iterativi: Jacobi, Gauss-Seidel, Gradiente (Steepest Descent) e Gradiente Coniugato. Il codice, corredato da un'interfaccia a riga di comando e da una moderna interfaccia utente testuale (TUI) realizzata con la libreria **Textual**, è stato testato su matrici sparse reali. Le prestazioni sono state confrontate in termini di iterazioni, tempo di esecuzione ed accuratezza per tolleranze comprese tra 10^{-2} e 10^{-10} . In questo resoconto vengono illustrati i fondamenti matematici, le considerazioni sull'aritmetica in virgola mobile, le scelte implementative, i risultati sperimentali e le conclusioni tratte dall'analisi.

Repository GitHub: https://github.com/deltanicolas/Progetto_Metodi_del_Calcolo_Scientifico_Unimib

Autori: Nicolas Chines (deltanicolas), Jacopo Borgato

Indice

1	Introduzione	3
2	Fondamenti matematici	3
2.1	Proprietà delle matrici SPD	3
2.2	Jacobi	3
2.3	Gauss-Seidel	4
2.4	Forma quadratica equivalente	4
2.5	Gradiente (Steepest Descent)	5
2.6	Gradiente Coniugato	5
3	Aritmetica in virgola mobile e precisione numerica	6
3.1	Rappresentazione IEEE 754 e machine epsilon	6
3.2	Scelta del range di tolleranze	6
4	Implementazione	6
4.1	Scelte implementative rilevanti	7
4.2	Interfaccia a riga di comando	8
5	Interfaccia utente testuale (TUI)	8
6	Matrici di test	8
7	Risultati sperimentali	9
7.1	Confronto complessivo	9
7.2	Grafici comparativi	9
8	Conclusioni	11
A	Istruzioni per l'esecuzione	12
A.1	Requisiti	12
A.2	Esecuzione di un singolo test (CLI)	12
A.3	Interfaccia TUI	12
A.4	Benchmark completo	12

1 Introduzione

La risoluzione di sistemi lineari di grandi dimensioni è un problema centrale nel calcolo scientifico. Quando la matrice dei coefficienti è simmetrica e definita positiva (SPD), si possono utilizzare metodi iterativi che sfruttano tale struttura per convergere rapidamente verso la soluzione. L'obiettivo del progetto è implementare e validare una libreria che offra quattro diversi solutori iterativi:

- Jacobi
- Gauss-Seidel
- Gradiente (Steepest Descent)
- Gradiente Coniugato

Tutti i metodi partono da un vettore iniziale nullo ($x^{(0)} = 0$) e usano come criterio di arresto il residuo relativo $\frac{\|Ax-b\|_2}{\|b\|_2} < \text{tol}$.

La libreria è stata testata su quattro matrici sparse reali fornite (SPA1, SPA2, VEM1, VEM2) e i risultati sono stati analizzati al variare della tolleranza ($10^{-2}, 10^{-4}, 10^{-6}, 10^{-8}, 10^{-10}$), mostrando i tipici comportamenti di convergenza e le prestazioni computazionali dei diversi algoritmi.

2 Fondamenti matematici

Sia $A \in \mathbb{R}^{n \times n}$ una matrice SPD e $b \in \mathbb{R}^n$ il termine noto. Vogliamo risolvere $Ax = b$. I metodi iterativi generano una successione $\{x^{(k)}\}$ che, sotto opportune condizioni, converge alla soluzione esatta x^* .

2.1 Proprietà delle matrici SPD

Una matrice A è *simmetrica e definita positiva* (SPD) se $A = A^T$ e $x^T A x > 0$ per ogni $x \neq 0$. Questa struttura garantisce che tutti gli autovalori $\lambda_i > 0$. Il *numero di condizionamento* in norma 2 è:

$$\kappa(A) = \frac{\lambda_{\max}}{\lambda_{\min}}, \quad (1)$$

ed è il parametro chiave che governa la velocità di convergenza di tutti i metodi iterativi: un κ elevato implica convergenza lenta e maggior sensibilità agli errori di arrotondamento.

Per le matrici SPD vale inoltre la decomposizione di Cholesky $A = LL^T$, che non viene utilizzata direttamente in questa libreria ma fornisce una base teorica per la garantita convergenza di Gauss-Seidel.

2.2 Jacobi

Il metodo di Jacobi scompone $A = D - L - U$, dove $D = \text{diag}(A)$, $-L$ è la parte triangolare inferiore stretta, $-U$ quella superiore. L'iterazione è:

$$x^{(k+1)} = D^{-1}(b + (L + U)x^{(k)}). \quad (2)$$

In pratica, sfruttando il residuo $r^{(k)} = b - Ax^{(k)}$, si può riscrivere come:

$$x^{(k+1)} = x^{(k)} + D^{-1}r^{(k)}, \quad (3)$$

che è la forma implementata in `solvers.py`. Il metodo converge se il raggio spettrale della matrice di iterazione $\rho(D^{-1}(L + U)) < 1$, condizione garantita, ad esempio, dalla stretta dominanza diagonale. Per matrici SPD la convergenza non è garantita in generale: è necessario che $2D - A$ sia definita positiva.

2.3 Gauss-Seidel

Il metodo di Gauss-Seidel utilizza la stessa decomposizione ma aggiorna le componenti in sequenza, sfruttando subito i valori appena calcolati. Detto $(D - L)$ la parte triangolare inferiore di A (diagonale inclusa) e $U = A - (D - L)$ la parte strettamente superiore, la formula di aggiornamento è:

$$x^{(k+1)} = (D - L)^{-1}(b - Ux^{(k)}). \quad (4)$$

Questa è la formulazione *diretta*, adottata nell'implementazione di riferimento: ad ogni iterazione si calcola il termine noto $b - Ux^{(k)}$ e si risolve il sistema triangolare inferiore $(D - L)x^{(k+1)} = b - Ux^{(k)}$ mediante sostituzione in avanti.

Formulazione equivalente via residuo. Nell'implementazione adottata si lavora sulla correzione $\delta^{(k)} = x^{(k+1)} - x^{(k)}$. Ad ogni iterazione si calcola il residuo corrente

$$r^{(k)} = b - Ax^{(k)}, \quad (5)$$

e si determina $\delta^{(k)}$ mediante sostituzione in avanti sul sistema triangolare inferiore $(D - L)\delta = r^{(k)}$, ovvero componente per componente:

$$\delta_i^{(k)} = \frac{r_i^{(k)} - \sum_{j=1}^{i-1} A_{ij} \delta_j^{(k)}}{A_{ii}}, \quad i = 1, \dots, n, \quad (6)$$

dove i $\delta_j^{(k)}$ con $j < i$ sono già stati calcolati nella stessa iterazione k . La soluzione viene quindi aggiornata come $x^{(k+1)} = x^{(k)} + \delta^{(k)}$. Questa formulazione è algebricamente equivalente a (4): sottraendo $x^{(k)}$ da entrambi i membri e moltiplicando per $(D - L)$ si ottiene infatti $(D - L)\delta^{(k)} = b - Ax^{(k)} = r^{(k)}$.

2.4 Forma quadratica equivalente

Per una matrice SPD il problema $Ax = b$ è equivalente alla minimizzazione della forma quadratica:

$$f(x) = \frac{1}{2}x^T Ax - b^T x. \quad (7)$$

Il gradiente di f è $\nabla f(x) = Ax - b$, che si annulla esattamente in $x^* = A^{-1}b$. Questa equivalenza è alla base dei metodi del gradiente e del gradiente coniugato.

2.5 Gradiente (Steepest Descent)

Il metodo del gradiente discende la forma quadratica $f(x)$ nella direzione opposta al gradiente locale, che coincide con il *residuo* $r^{(k)} = b - Ax^{(k)}$. Il passo α_k è scelto in modo ottimale lungo tale direzione:

$$r^{(k)} = b - Ax^{(k)}, \quad (8)$$

$$\alpha_k = \frac{(r^{(k)})^T r^{(k)}}{(r^{(k)})^T A r^{(k)}}, \quad (9)$$

$$x^{(k+1)} = x^{(k)} + \alpha_k r^{(k)}. \quad (10)$$

La convergenza è lineare con fattore $\left(\frac{\kappa-1}{\kappa+1}\right)^2$, che può essere molto vicino a 1 per matrici mal condizionate, causando il noto fenomeno di *zig-zag*: direzioni consecutive sono ortogonali tra loro, con conseguente progresso molto lento verso la soluzione.

2.6 Gradiente Coniugato

Il metodo del Gradiente Coniugato (CG) è un'estensione del metodo del gradiente per sistemi SPD. L'idea di base è la stessa: muoversi lungo direzioni che riducono l'errore. Il problema del gradiente puro è che tende a "zigzagare", ripercorrendo direzioni già esplorate e rallentando la convergenza.

Il CG corregge questo difetto imponendo che ogni nuova direzione $d^{(k)}$ sia *coniugata* rispetto alle precedenti, cioè $(d^{(i)})^T A d^{(j)} = 0$ per $i \neq j$: in questo modo nessun passo "disfa" il lavoro dei passi precedenti. La direzione viene costruita combinando il residuo corrente con la direzione precedente tramite un coefficiente β_k :

$$r^{(0)} = b - Ax^{(0)}, \quad d^{(0)} = r^{(0)}, \quad (11)$$

$$\alpha_k = \frac{\|r^{(k)}\|^2}{(d^{(k)})^T A d^{(k)}}, \quad (12)$$

$$x^{(k+1)} = x^{(k)} + \alpha_k d^{(k)}, \quad (13)$$

$$r^{(k+1)} = r^{(k)} - \alpha_k A d^{(k)}, \quad (14)$$

$$\beta_k = \frac{\|r^{(k+1)}\|^2}{\|r^{(k)}\|^2}, \quad (15)$$

$$d^{(k+1)} = r^{(k+1)} + \beta_k d^{(k)}. \quad (16)$$

Grazie alla coniugazione, il metodo converge al più in n iterazioni in aritmetica esatta. Nella pratica converge molto prima: il tasso di riduzione dell'errore nella norma A è

$$\frac{\|e^{(k)}\|_A}{\|e^{(0)}\|_A} \leq 2 \left(\frac{\sqrt{\kappa} - 1}{\sqrt{\kappa} + 1} \right)^k, \quad (17)$$

dove $\kappa = \lambda_{\max}/\lambda_{\min}$ è il numero di condizione della matrice. Rispetto allo steepest descent, dove la convergenza dipende da κ , qui dipende da $\sqrt{\kappa}$: per matrici mal condizionate la differenza è sostanziale.

3 Aritmetica in virgola mobile e precisione numerica

3.1 Rappresentazione IEEE 754 e machine epsilon

Tutti i calcoli della libreria avvengono in doppia precisione (`float64`), come da standard IEEE 754. In questo formato ogni numero è rappresentato con 53 bit di mantissa, dando una precisione relativa di:

$$\varepsilon_{\text{mach}} = 2^{-52} \approx 2.22 \times 10^{-16}. \quad (18)$$

Questa quantità rappresenta il più piccolo numero tale che $1 + \varepsilon_{\text{mach}} \neq 1$ in aritmetica floating point. Essa costituisce il *limite assoluto* di accuratezza per qualunque calcolo in `float64`: non ha senso richiedere a un metodo iterativo una tolleranza inferiore a $\varepsilon_{\text{mach}}$, in quanto il criterio di arresto verrebbe valutato su valori che sono essi stessi affetti da errori di arrotondamento di quell'ordine.

3.2 Scelta del range di tolleranze

Per questa ragione il benchmark è stato eseguito con tolleranze nell'intervallo $[10^{-2}, 10^{-10}]$, ben al di sopra di $\varepsilon_{\text{mach}}$:

$$\text{tol} \in \{10^{-2}, 10^{-4}, 10^{-6}, 10^{-8}, 10^{-10}\}. \quad (19)$$

Richiedere una tolleranza di, ad esempio, 10^{-16} o 10^{-17} sarebbe problematico per due motivi distinti:

1. Il criterio $\|Ax - b\|_2 / \|b\|_2 < 10^{-16}$ non potrebbe mai essere soddisfatto a causa degli arrotondamenti accumulati nei prodotti matrice-vettore, e il metodo itererebbe fino all'esaurimento del numero massimo di iterazioni senza convergere.
2. Anche se il residuo raggiungesse tale ordine di grandezza, l'errore effettivo sulla soluzione sarebbe amplificato dal numero di condizionamento: per una matrice con $\kappa(A)$, un residuo relativo di 10^{-16} non garantisce affatto un errore sulla soluzione di 10^{-16} , ma potenzialmente di $\kappa(A) \cdot 10^{-16}$.

La relazione tra residuo relativo ed errore relativo sulla soluzione è:

$$\frac{\|x^* - x\|_2}{\|x^*\|_2} \leq \kappa(A) \cdot \frac{\|Ax - b\|_2}{\|b\|_2}. \quad (20)$$

Questo implica che, per matrici mal condizionate, il criterio di arresto basato sul residuo può essere raggiunto quando la soluzione approssimata è ancora lontana da quella esatta.

4 Implementazione

La libreria è scritta in Python e sfrutta NumPy, SciPy (per il formato sparse CSR e la lettura di file Matrix Market) e Matplotlib per i grafici. Il codice è organizzato in moduli separati:

- `solvers.py` : contiene la classe `MatSolvers` con i quattro metodi iterativi. Ogni metodo riceve la matrice A , il vettore b e la tolleranza, e restituisce la soluzione approssimata, il numero di iterazioni, un flag di convergenza e il tempo di esecuzione.

- `utility.py` : funzioni per il caricamento di matrici `.mtx`, la costruzione del sistema di test ($x_{\text{exact}} = \mathbf{1}$, $b = Ax_{\text{exact}}$) e il calcolo dell'errore relativo.
- `test.py` : interfaccia unificata per eseguire un solutore su una data matrice e tolleranza; usata sia dalla CLI che dal benchmark.
- `main.py` : interfaccia a riga di comando che permette di scegliere file, tolleranza e metodo.
- `argparser.py` : gestione e validazione degli argomenti da terminale.
- `benchmark.py` : esegue tutti i solutori su tutte le matrici `.mtx` nella cartella `Data` per una gamma di tolleranze ($10^{-2}, 10^{-4}, 10^{-6}, 10^{-8}, 10^{-10}$) e salva i risultati in un file CSV.
- `app.py` : interfaccia utente testuale interattiva (TUI) realizzata con la libreria `Textual`.
- `plotter.py` : legge il CSV e genera, per ciascuna matrice, due grafici (tempo vs tolleranza, iterazioni vs tolleranza) confrontando i quattro metodi.

4.1 Scelte implementative rilevanti

Formato sparse. Le matrici, lette con `scipy.io.mmread`, sono convertite in formato CSR (`csr_matrix`) per ottimizzare i prodotti matrice-vettore e l'accesso alle diagonal. Tutti i prodotti Av nei solutori vengono eseguiti tramite `A.dot(v)`, sfruttando la memorizzazione sparse.

Gauss-Seidel e densificazione parziale. L'implementazione di Gauss-Seidel è l'unica a richiedere la parte triangolare inferiore densa $L = \text{np.tril}(A.toarray())$, necessaria per il ciclo componente per componente. Questa scelta può diventare costosa in termini di memoria per matrici di dimensioni molto elevate ($n \gg 10^4$), dove sarebbe preferibile una risoluzione triangolare sparse con `scipy.sparse.linalg.spsolve_triangular`. Per le matrici del dataset fornito la densificazione è accettabile.

Criterio di arresto. $\|Ax - b\|_2 / \|b\|_2 < \text{tol}$, calcolato dopo ogni iterazione completa. Il residuo relativo fornisce un controllo diretto e scalabile, indipendente dalla norma di b .

Guess iniziale. $x^{(0)} = \mathbf{0}$.

Sistema di test. Per ogni matrice viene costruito un sistema con soluzione esatta $x^* = (1, 1, \dots, 1)^T$ e termine noto $b = Ax^*$. In questo modo l'errore relativo $\frac{\|x^* - x^{(k)}\|_2}{\|x^*\|_2}$ è sempre calcolabile e confrontabile.

Numero massimo di iterazioni. Fissato a 20000 per Jacobi, Gauss-Seidel e Gradiente; per il Gradiente Coniugato è posto a n (dimensione della matrice), come limite teorico, ma di fatto la tolleranza viene raggiunta molto prima.

4.2 Interfaccia a riga di comando

La sintassi principale è:

```
1 python main.py -f <file.mtx> -t <tolleranza> [-s <id>]
```

L'opzione `-s` accetta un intero da 0 a 4: 0 esegue tutti i metodi, 1 Jacobi, 2 Gauss-Seidel, 3 Gradiente, 4 Gradiente Coniugato. Esempio:

```
1 python main.py -f Data/spa1.mtx -t 1e-6 -s 4
```

esegue il Gradiente Coniugato sulla matrice `spa1.mtx` con tolleranza 10^{-6} .

5 Interfaccia utente testuale (TUI)

In aggiunta alla CLI, il progetto include `app.py`, un'interfaccia utente testuale interattiva (*Terminal User Interface*) realizzata con la libreria Python `Textual`. L'obiettivo è offrire un modo visivo, comodo e immediato per eseguire i risolutori più volte e con differenti parametri.



Figura 1: Interfaccia Grafica da Terminale TUI.

6 Matrici di test

Il dataset fornito comprende quattro matrici sparse in formato Matrix Market (`.mtx`):

- `spa1.mtx`, `spa2.mtx` : matrici di dimensioni relativamente contenute, con struttura a bande e buone proprietà spettrali.
- `vem1.mtx`, `vem2.mtx` : matrici provenienti da discretizzazioni agli elementi virtuali (VEM), di dimensioni maggiori e con spettro più ampio e numero di condizionamento più elevato.

Tutte sono simmetriche e definite positive, e presentano un'elevata sparsità. La struttura spettrale influisce direttamente sulla velocità di convergenza di tutti i metodi iterativi, come evidenziato nei risultati sperimentali.

7 Risultati sperimentali

Il benchmark è stato eseguito per tolleranze decrescenti:

$$\text{tol} \in \{10^{-2}, 10^{-4}, 10^{-6}, 10^{-8}, 10^{-10}\}. \quad (21)$$

I grafici generati da `plotter.py` mostrano chiaramente le prestazioni relative dei quattro solutori. Di seguito vengono riportati e commentati i risultati principali.

7.1 Confronto complessivo

I risultati mostrano comportamenti molto diversi a seconda della matrice, e non esiste un ordinamento fisso tra i metodi valido in tutti i casi.

- **Gradiente Coniugato** è il metodo più robusto: mantiene un numero di iterazioni estremamente basso su tutte le matrici e domina nettamente su `vem1` e `vem2`, dove gli altri metodi degradano sensibilmente. È l'unica scelta affidabile al crescere del numero di condizionamento.
- **Gauss-Seidel** presenta un comportamento opposto a seconda della matrice. Su `spa1` e `spa2` è competitivo in termini di tempo — su `spa2` risulta addirittura più veloce del CG — grazie al basso costo per iterazione. Su `vem1` e `vem2` diventa invece il metodo *più lento* in assoluto: pur avendo meno iterazioni di Jacobi, il costo della sostituzione in avanti su struttura densa scala male con la dimensione, portando a tempi fino a un ordine di grandezza superiori agli altri.
- **Jacobi** è stabile e prevedibile: il numero di iterazioni cresce linearmente con la tolleranza e il costo per iterazione è basso. Su `vem1` e `vem2` è più lento di GRAD e CG in iterazioni, ma più veloce di GS in tempo grazie alla semplicità dell'aggiornamento.
- **Gradiente (Steepest Descent)** mostra il comportamento più anomalo: su `spa1` e `spa2` esplode completamente, con oltre 10 000 iterazioni a tolleranza 10^{-10} , probabilmente a causa di un forte comportamento a zig zag che lo porta ad oscillare attorno al minimo. Su `vem1` e `vem2`, nonostante un numero di condizionamento più alto, si comporta invece in modo simile a Jacobi, rimanendo nella stessa fascia di iterazioni e tempi.

7.2 Grafici comparativi

Le Figure 2, 3, 4 e 5 mostrano per ciascuna matrice l'evoluzione del tempo di calcolo e del numero di iterazioni al variare della tolleranza.

Su `spa1` (Figura 2) il Gradiente esplode in iterazioni mentre gli altri tre metodi rimangono comparabili in tempo; il CG è il più veloce ma il margine su GS e Jacobi è contenuto.

Su `spa2` (Figura 3) il quadro è analogo, con la differenza che GS risulta il più veloce in tempo grazie al basso numero di iterazioni combinato con un costo per iterazione ancora gestibile sulla dimensione di questa matrice.

Su `vem1` (Figura 4) il CG è nettamente separato dagli altri (circa 1–3 ms contro decine o centinaia), mentre GS diventa il *peggiore* in tempo nonostante abbia meno iterazioni di Jacobi, evidenziando come il costo della sostituzione in avanti diventi dominante.

Su **vem2** (Figura 5) il distacco di CG si accentua ulteriormente e GS raggiunge tempi dell'ordine dei 10 000 ms, peggio di tutti gli altri metodi. Jacobi e GRAD si comportano in modo simile sia in iterazioni che in tempo.

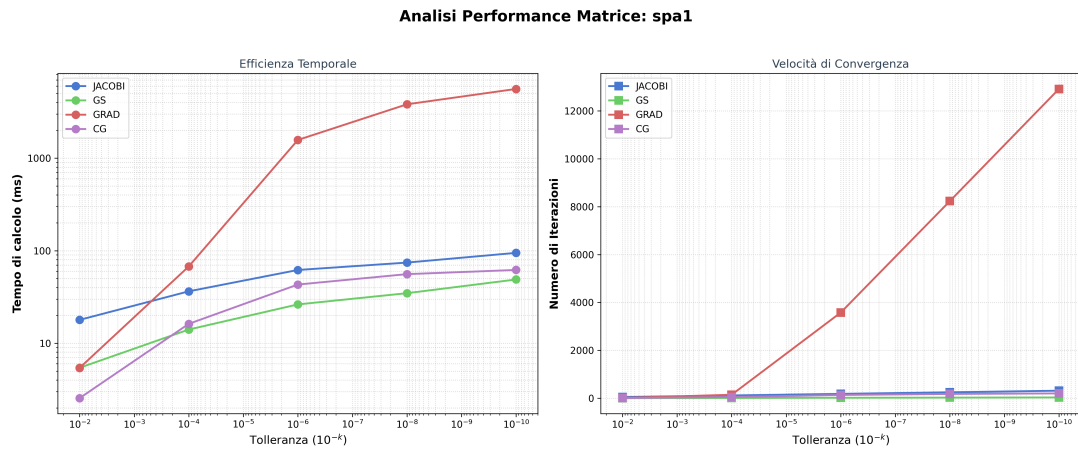


Figura 2: Matrice SPA1: tempo di calcolo e numero di iterazioni al variare della tolleranza.

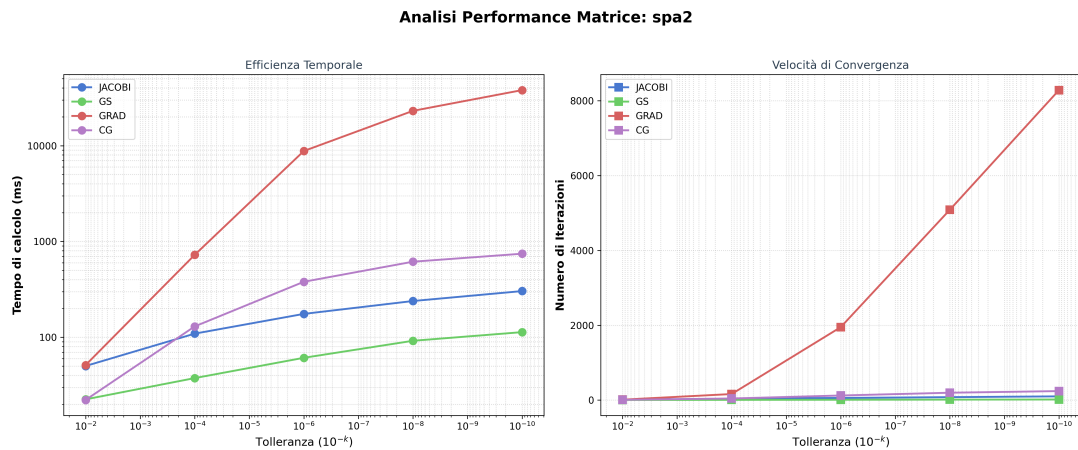


Figura 3: Matrice SPA2: tempo di calcolo e numero di iterazioni al variare della tolleranza.

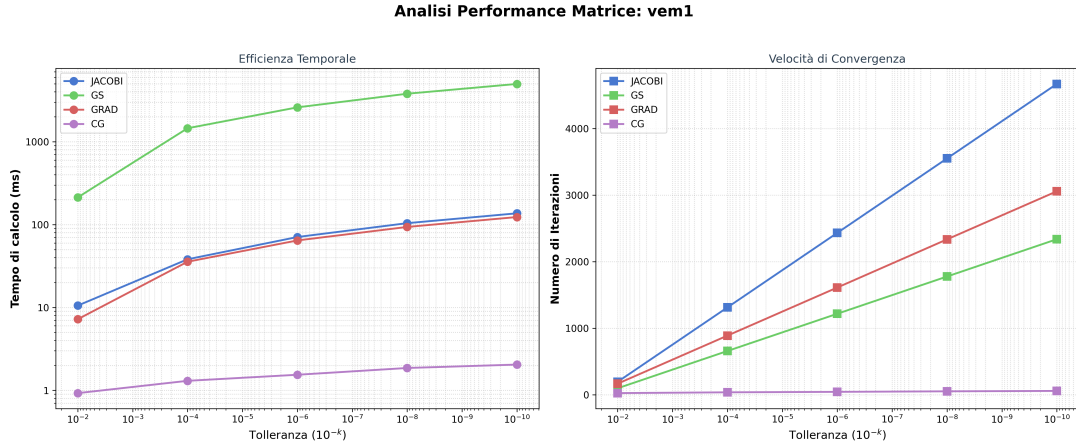


Figura 4: Matrice VEM1: tempo di calcolo e numero di iterazioni al variare della tolleranza.

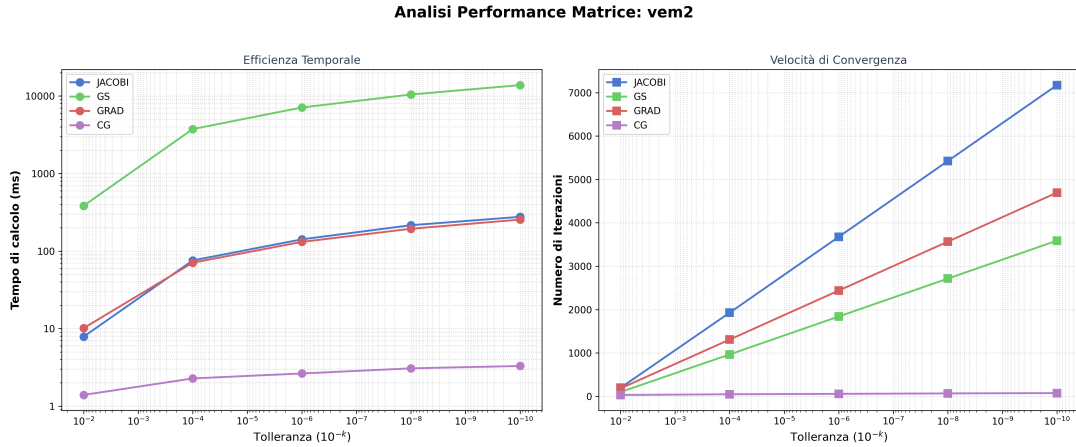


Figura 5: Matrice VEM2: tempo di calcolo e numero di iterazioni al variare della tolleranza.

8 Conclusioni

Il progetto ha raggiunto l'obiettivo di implementare e confrontare quattro solutori iterativi per sistemi lineari SPD, affiancati da un'interfaccia a riga di comando e da una TUI moderna. L'architettura modulare del codice e lo script di benchmark hanno permesso un'analisi sistematica delle prestazioni al variare di matrice e tolleranza.

I risultati confermano la superiorità del Gradiente Coniugato, che unisce velocità di convergenza e stabilità numerica, risultando la scelta ottimale per sistemi SPD di grandi dimensioni. I metodi di Jacobi e Gauss-Seidel, pur semplici da implementare, richiedono molte più iterazioni; Gauss-Seidel è generalmente preferibile a Jacobi, ma l'implementazione naive su formato denso ne limita la scalabilità. Il metodo del Gradiente, pur convergente, soffre del fenomeno di zig-zag su problemi mal condizionati.

L'attività ha permesso di approfondire non solo gli aspetti teorici dei metodi iterativi, ma anche di vedere in modo pratico come la scelta del solutore non sia mai indifferente: matrici con struttura e condizionamento diversi premiano metodi diversi, e affidarsi a un

unico algoritmo senza analizzarne le proprietà può portare a prestazioni peggiori, come dimostrato da Gauss-Seidel su `vem2` o dal Gradiente su `spa1`.

A Istruzioni per l'esecuzione

A.1 Requisiti

```
1 pip install -r requirements.txt
```

Il file `requirements.txt` include NumPy, SciPy, Matplotlib, Pandas e Textual.

A.2 Esecuzione di un singolo test (CLI)

```
1 python main.py -f Data/spa1.mtx -t 1e-6 -s 4
```

A.3 Interfaccia TUI

```
1 python app.py
```

Utilizzare `Ctrl+R` per eseguire, `Ctrl+C` per pulire la console, `Ctrl+Q` per uscire.

A.4 Benchmark completo

```
1 python benchmark.py
2 python plotter.py
```

I risultati numerici vengono salvati in `Data/benchmark_results.csv` e i grafici nella cartella `Data/Plot/`.